keyword. The ECMAScript 6 standard does not specify the single-arrow function but it does have the fat arrow:

```
let square = (x) => x * x                    <-- ECMAScript 6 example
```

You would be forgiven for mistaking that for CoffeeScript. That's because it was *inspired* by CoffeeScript syntax—the relationship between CoffeeScript and ECMAScript is *symbiotic*.

That's it for the familiar things. How about all the unfamiliar features that you need to know about to stay in the game? To begin with, what's in ECMAScript 5 that a CoffeeScripter needs to find out about?

13.2. ECMAScript 5

Released in December 2009, the primary focus for the changes in ECMAScript 5 was to improve robustness for the language and, as a result, for runtimes and programs. The key features to be aware of are native JSON support, strict mode, property descriptors, and changes to the `Object` object. The best way to learn them is to try them, which means your runtime needs to support them.

13.2.1. Runtime support

It's easy to play with the ECMAScript 5 features discussed in this section; simply fire up the CoffeeScript REPL. All of these features are supported in the V8 JavaScript engine that powers Node.js, your REPL, and all of your server-side CoffeeScript programs. Unfortunately, the same does not hold true for all of your browser-based programs. Some browsers don't support all of these features, so if you need to support those browsers, you have to either avoid those features or use a polyfill (as discussed in chapter 11) where possible.

Polyfills